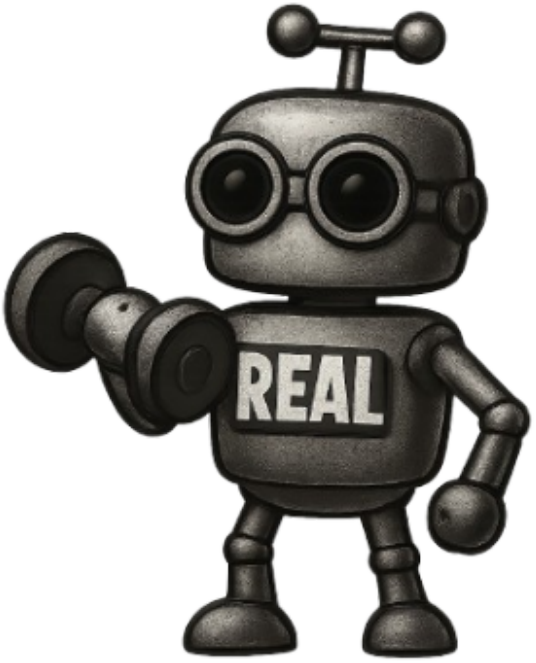
IES Albarregas
Avda. Felipe Corchero, 37
06800 (Mérida)
924 00 98 56
ies.albarregas@edu.juntaex.es

JUNTA DE EXTREMADURA
Consejería de Educación, Ciencia y Formación Profesional



RealSteel

Por :
Carlos De Tena y Fernando Romero Gil

Manual Técnico

1. INTRODUCCIÓN

RealSteel es una aplicación móvil orientada al ámbito del fitness cuyo objetivo principal es facilitar la planificación, seguimiento y registro de entrenamientos de manera digital. La aplicación permite a los usuarios organizar sus rutinas, registrar pesos y repeticiones, así como consultar ejercicios clasificados por grupo muscular.

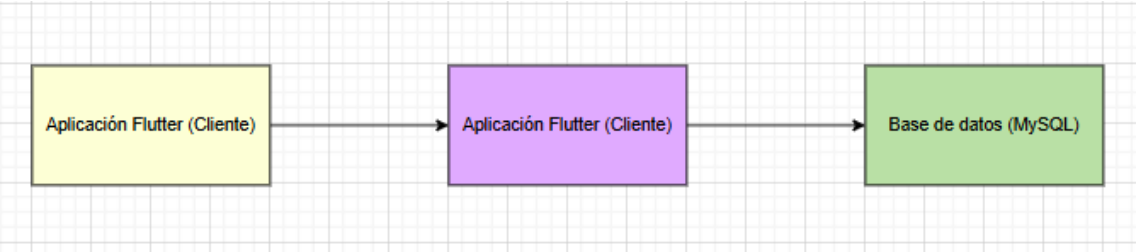
Este proyecto surge como respuesta a la necesidad de sustituir métodos tradicionales poco estructurados (como notas en papel o conversaciones en aplicaciones de mensajería) por una herramienta digital eficiente, intuitiva y visual.

El desarrollo del proyecto se ha llevado a cabo utilizando una arquitectura cliente-servidor, separando claramente el frontend, backend y base de datos, lo que permite una mayor escalabilidad, mantenibilidad y claridad en la estructura del sistema.

2. ARQUITECTURA DE LA APLICACIÓN

2.1 Visión general

La aplicación RealSteel sigue una arquitectura de tipo cliente-servidor, en la que cada componente tiene una responsabilidad claramente definida.



Este modelo permite separar completamente la lógica de presentación, la lógica de negocio y la persistencia de datos.

2.2 Backend

El backend ha sido desarrollado utilizando **Spring Boot**, un framework ampliamente utilizado para la creación de aplicaciones Java basadas en servicios REST.

Se ha estructurado siguiendo una arquitectura en capas:

- **Controller**: Gestiona las peticiones HTTP (endpoints)
- **Service**: Contiene la lógica de negocio
- **Repository (JPA)**: Acceso a la base de datos
- **Entity**: Representación de las tablas de la base de datos

Esta separación permite mejorar la organización del código y facilitar su mantenimiento.

2.3 Frontend

El frontend ha sido desarrollado con **Flutter**, lo que permite crear una aplicación móvil multiplataforma con una única base de código.

Características principales del frontend:

- Interfaz moderna y visual
- Navegación entre pantallas (login, menú, ejercicios...)
- Consumo de API REST mediante peticiones HTTP
- Uso de widgets reutilizables

2.4 Comunicación cliente-servidor

La comunicación entre frontend y backend se realiza mediante peticiones HTTP en formato JSON.

Ejemplo de flujo:

1. El usuario inicia sesión en Flutter
2. Flutter envía una petición POST a la API
3. El backend valida los datos
4. Devuelve una respuesta en formato JSON
5. Flutter procesa la respuesta y actualiza la UI

3. TECNOLOGÍAS UTILIZADAS (EXPLICACIÓN DETALLADA)

El desarrollo de la aplicación RealSteel se ha basado en el uso de diferentes tecnologías que permiten construir un sistema completo siguiendo una arquitectura cliente-servidor. Cada una de estas tecnologías ha sido seleccionada en función de su eficiencia, compatibilidad y facilidad de integración dentro del proyecto.

A continuación, se describen en detalle las tecnologías utilizadas y el papel que desempeñan dentro del sistema.

3.1 Frontend

Flutter

Flutter es un framework de desarrollo multiplataforma desarrollado por Google que permite crear aplicaciones móviles utilizando un único código base.

En el proyecto RealSteel, Flutter se ha utilizado para implementar la interfaz de usuario, encargándose de toda la interacción con el usuario final.

Funciones dentro del proyecto

- Desarrollo de las diferentes pantallas de la aplicación (inicio de sesión, menú principal, listado de ejercicios, etc.).
- Gestión de la navegación entre vistas.
- Consumo de la API REST mediante peticiones HTTP.
- Representación visual de los datos obtenidos desde el backend.
- Organización de la interfaz mediante widgets reutilizables.

Ventajas

- Permite el desarrollo multiplataforma (Android e iOS) con una sola base de código.
- Ofrece un alto rendimiento gracias a la compilación nativa.
- Dispone de una amplia biblioteca de componentes visuales.
- Facilita la creación de interfaces modernas y dinámicas.

Dart

Dart es el lenguaje de programación utilizado por Flutter. Se trata de un lenguaje orientado a objetos que permite desarrollar aplicaciones estructuradas y eficientes.

Función en el proyecto

- Implementación de la lógica de la aplicación.
- Definición de modelos de datos (por ejemplo, User o Exercise).
- Desarrollo de servicios para la comunicación con la API.
- Gestión del estado y de los eventos de la interfaz.

Características

- Tipado fuerte que reduce errores en tiempo de ejecución.

- Sintaxis clara y moderna.
- Integración directa con el framework Flutter.

3.2 Backend

Java

Java es el lenguaje de programación utilizado para el desarrollo del backend. Se ha seleccionado por su robustez, estabilidad y amplio uso en entornos empresariales.

Función en el proyecto

- Implementación de la lógica de negocio.
- Definición de las entidades del sistema.
- Gestión de las peticiones HTTP.

Spring Boot

Spring Boot es un framework basado en Java que facilita la creación de aplicaciones backend y servicios REST de forma rápida y estructurada.

Función en el proyecto

- Desarrollo de la API REST.
- Gestión de endpoints mediante controladores.
- Procesamiento de peticiones y respuestas HTTP.
- Integración con la base de datos a través de JPA.

Arquitectura en capas

El backend se ha organizado siguiendo una arquitectura en capas:

- Controller: define los endpoints de la API.
- Service: contiene la lógica de negocio.

- Repository: gestiona el acceso a la base de datos.
- Entity: representa las tablas de la base de datos.

Esta estructura mejora la mantenibilidad del código y facilita futuras ampliaciones.

JPA / Hibernate

JPA (Java Persistence API) es una especificación que permite gestionar la persistencia de datos en aplicaciones Java. En este proyecto se utiliza Hibernate como implementación de JPA.

Función en el proyecto

- Mapear las entidades Java a tablas de la base de datos.
- Realizar operaciones CRUD sin necesidad de escribir consultas SQL manualmente.
- Gestionar las relaciones entre entidades.

Ventajas

- Reduce la cantidad de código necesario para acceder a datos.
- Abstrae la interacción con la base de datos.
- Facilita el trabajo con relaciones complejas entre entidades.

3.3 Base de datos

MySQL

MySQL es un sistema gestor de bases de datos relacional ampliamente utilizado. En RealSteel se emplea para almacenar toda la información persistente del sistema.

Función en el proyecto

- Almacenamiento de datos de usuarios, rutinas, ejercicios y registros.
- Gestión de relaciones entre entidades mediante claves foráneas.
- Soporte para consultas realizadas desde el backend.

Características

- Alta fiabilidad y rendimiento.
- Compatibilidad con JPA/Hibernate.
- Soporte para integridad referencial.

3.4 Herramientas utilizadas

Git y GitHub

Git es un sistema de control de versiones distribuido, mientras que GitHub es una plataforma para alojar repositorios.

Función en el proyecto

- Control de versiones del código.
- Trabajo colaborativo entre los desarrolladores.
- Seguimiento de cambios y evolución del proyecto.
-

Postman

Postman es una herramienta utilizada para probar APIs REST.

Función en el proyecto

- Verificación del funcionamiento de los endpoints.
- Pruebas de peticiones HTTP (GET y POST).
- Validación de respuestas en formato JSON.

Figma

Figma es una herramienta de diseño de interfaces que permite crear prototipos visuales de aplicaciones.

Función en el proyecto

- Diseño de la interfaz de usuario.
- Prototipado de pantallas antes del desarrollo.
- Planificación visual de la experiencia de usuario.

4 ENTIDADES PRINCIPALES (EXPLICACIÓN DETALLADA)

El sistema RealSteel se basa en un modelo de datos relacional compuesto por varias entidades que representan los elementos principales de la aplicación. Estas entidades permiten estructurar la información de manera organizada, garantizando la integridad de los datos y facilitando su gestión.

A continuación, se describen en detalle cada una de las entidades y sus atributos.

ENTIDAD USUARIO

La entidad **Usuario** representa a cada persona que utiliza la aplicación. Es una de las entidades principales del sistema, ya que todas las demás entidades están directa o indirectamente relacionadas con ella.

Cada usuario puede crear rutinas, registrar entrenamientos y gestionar su propio progreso.

Atributos

- **id**
  Identificador único del usuario dentro del sistema.
  Se trata de una clave primaria generada automáticamente, que permite distinguir a cada usuario de forma inequívoca.

- **name**
  Nombre del usuario.
  Se utiliza para identificarlo dentro de la aplicación y mostrarlo en la interfaz.

- **email**
  Correo electrónico del usuario.
  Este campo debe ser único, ya que se utiliza como credencial principal para el inicio de sesión.

- **password**
  Contraseña del usuario.
  Se emplea para la autenticación en el sistema. En un entorno real, debería almacenarse cifrada por motivos de seguridad.

Relación

Un usuario puede tener múltiples rutinas (relación 1:N).

ENTIDAD RUTINA

La entidad **Rutina** representa un conjunto de ejercicios organizados por el usuario para estructurar sus entrenamientos.

Permite a los usuarios planificar sus sesiones y dividir sus entrenamientos por días o grupos musculares.

Atributos

- **id**
  Identificador único de la rutina.

Clave primaria que permite diferenciar cada rutina.

- **name**
  Nombre de la rutina.
  Permite al usuario identificar fácilmente el tipo de entrenamiento (por ejemplo: "Pecho", "Pierna", "Full Body").

- **description**
  Descripción de la rutina.
  Campo opcional donde el usuario puede añadir información adicional sobre el entrenamiento.

- **user_id**
  Identificador del usuario al que pertenece la rutina.
  Es una clave foránea que establece la relación entre la rutina y el usuario.

Relación

- Muchas rutinas pertenecen a un usuario (N:1)

- Una rutina puede contener varios ejercicios (1:N)

ENTIDAD EJERCICIO

La entidad **Ejercicio** representa cada uno de los ejercicios disponibles en la aplicación.

Estos ejercicios forman parte del catálogo que el usuario puede consultar y utilizar dentro de sus rutinas.

Atributos

- **id**
  Identificador único del ejercicio.
  Clave primaria del sistema.

- **name**
  Nombre del ejercicio.

Ejemplo: "Press banca", "Sentadilla", "Curl bíceps".

- **muscle**
  Grupo muscular principal trabajado.
  Este atributo permite clasificar los ejercicios y facilitar su búsqueda o filtrado.

- **equipment**
  Equipamiento necesario para realizar el ejercicio.
  Ejemplo: barra, mancuernas, máquina, peso corporal.

- **image**
  Ruta de la imagen asociada al ejercicio.
  Se utiliza en el frontend para mostrar visualmente el ejercicio al usuario, mejorando la experiencia de uso.

Relación

- Un ejercicio puede estar presente en múltiples rutinas

- Un ejercicio puede tener múltiples registros de entrenamiento

ENTIDAD REGISTRO (RECORD)

La entidad **Registro** (o Record) representa cada entrada de datos relacionada con el rendimiento del usuario durante un ejercicio.

Es una de las entidades más importantes, ya que permite realizar el seguimiento del progreso del usuario.

Atributos

- **id**
  Identificador único del registro.
  Clave primaria.

- **weight**
  Peso utilizado en el ejercicio.
  Permite medir la carga levantada por el usuario en cada sesión.

- **repetitions**
  Número de repeticiones realizadas.
  Indica cuántas veces se ha ejecutado el movimiento.

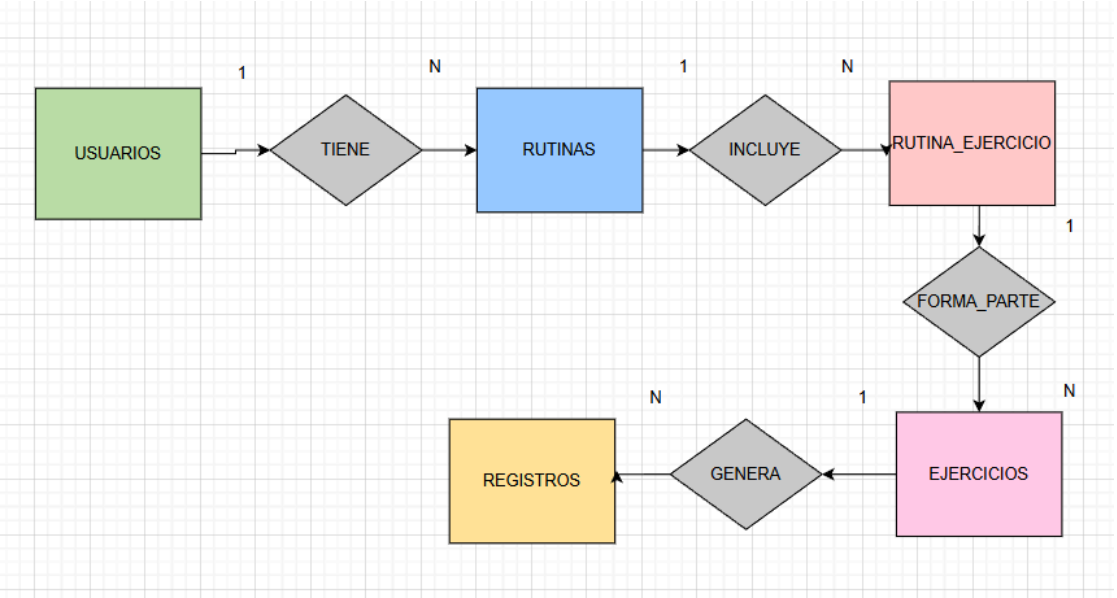
- **date**
  Fecha en la que se realizó el ejercicio.
  Permite llevar un control temporal del progreso.

- **exercise_id**
  Identificador del ejercicio al que pertenece el registro.
  Es una clave foránea que vincula el registro con un ejercicio concreto.

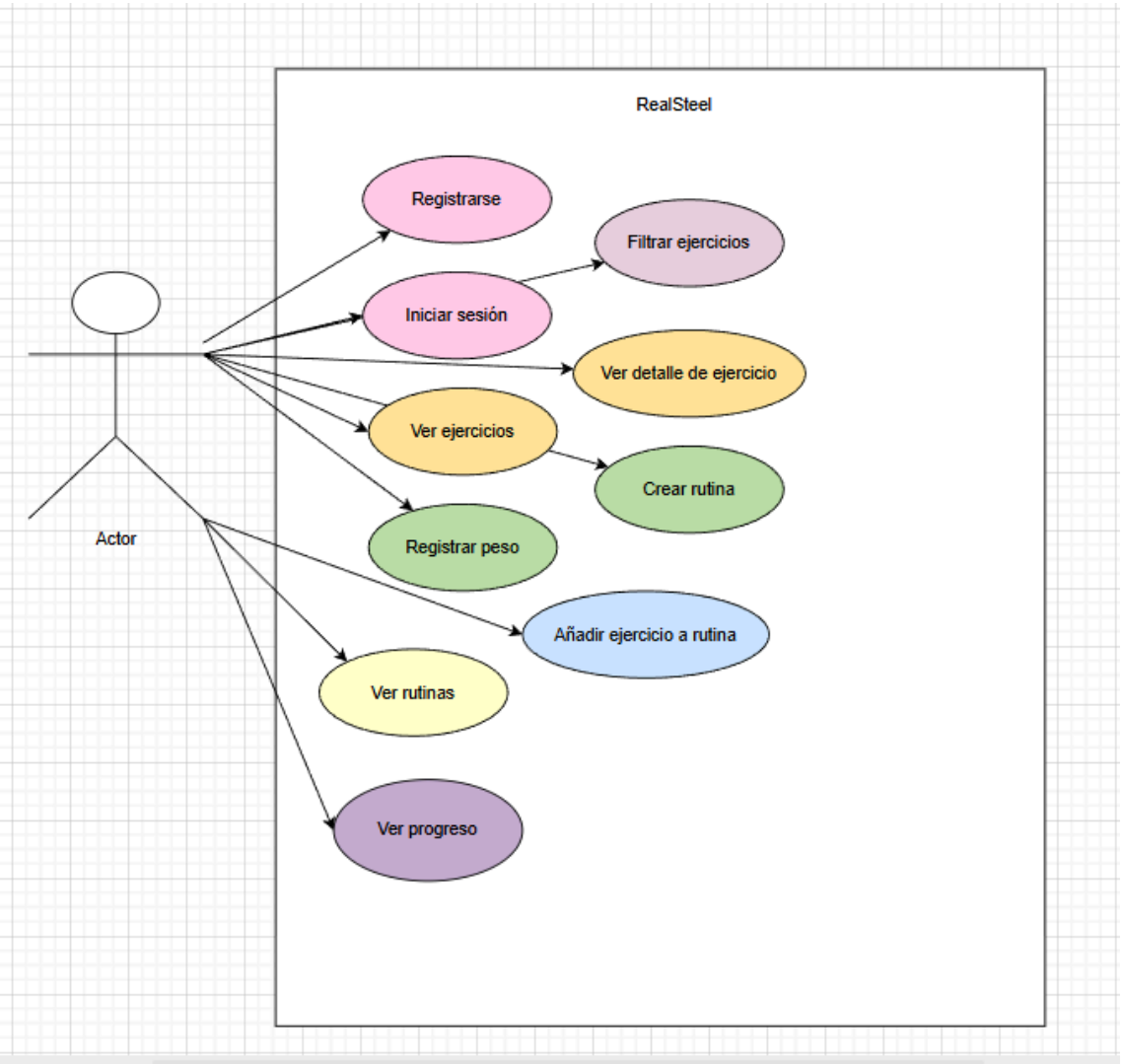
Relación

- Un ejercicio puede tener múltiples registros (1:N)

Modelo Entidad / Relación de la BBDD



Casos de uso


RealSteel
Registrarse
Filtrar ejercicios
Iniciar sesión
Ver detalle de ejercicio
Ver ejercicios
Crear rutina
Registrar peso
Actor
Añadir ejercicio a rutina
Ver rutinas
Ver progreso

5. API REST

5.1 Introducción

La API REST constituye el elemento central de comunicación entre el frontend desarrollado en Flutter y el backend implementado con Spring Boot. Su función principal es permitir el intercambio de información de forma estructurada mediante el protocolo HTTP, utilizando el formato JSON como estándar para la transmisión de datos.

El uso de una arquitectura REST (Representational State Transfer) aporta múltiples ventajas al sistema, entre las que destacan la separación de responsabilidades, la escalabilidad y la facilidad de integración con otros servicios.

En este proyecto, la API actúa como intermediaria entre la lógica de negocio y la interfaz de usuario, permitiendo que la aplicación móvil consuma datos sin necesidad de acceder directamente a la base de datos.

5.2 Métodos HTTP utilizados

En la API desarrollada se han empleado principalmente dos tipos de métodos HTTP:

- **GET**: utilizado para obtener información del servidor. Este método no modifica los datos existentes y se utiliza para consultas.

- **POST**: utilizado para enviar datos al servidor con el objetivo de crear nuevos registros.

Aunque existen otros métodos como PUT o DELETE, en esta fase del proyecto (PMV) se ha optado por limitar la API a las operaciones esenciales.

5.3 Estructura de los endpoints

Los endpoints siguen una estructura clara y coherente, basada en nombres de recursos en plural y rutas jerárquicas. Esto facilita su comprensión y uso desde el frontend.

Usuarios

Los endpoints relacionados con los usuarios permiten gestionar el registro y la autenticación.

- **POST /users**
  Permite crear un nuevo usuario en el sistema. Recibe los datos en formato JSON y los almacena en la base de datos.

- **GET /users**
  Devuelve una lista con todos los usuarios registrados.

- **POST /users/login**
  Permite validar las credenciales del usuario y devolver su información en caso de éxito.

Rutinas

Estos endpoints permiten gestionar las rutinas de entrenamiento de los usuarios.

- **POST /routines**
  Crea una nueva rutina asociada a un usuario.

- **GET /routines/user/{id}**
  Obtiene todas las rutinas asociadas a un usuario concreto.

Ejercicios

- **GET /exercises**
  Devuelve el catálogo de ejercicios disponibles en la aplicación, incluyendo información como el grupo muscular o el equipamiento necesario.

Registros

- **POST /records**
  Permite guardar información sobre el rendimiento del usuario en

un ejercicio, como peso y repeticiones.

- **GET /records/exercise/{id}**
  Obtiene el historial de registros asociados a un ejercicio concreto, lo que permite analizar la evolución del usuario.

5.4 Flujo de comunicación

El funcionamiento de la API puede describirse mediante el siguiente flujo:

1. El usuario realiza una acción en la aplicación (por ejemplo, iniciar sesión).
2. Flutter envía una petición HTTP al endpoint correspondiente.
3. El backend procesa la solicitud mediante el controlador adecuado.
4. Se ejecuta la lógica de negocio en la capa service.
5. Se accede a la base de datos mediante JPA.
6. Se devuelve una respuesta en formato JSON.
7. El frontend interpreta la respuesta y actualiza la interfaz.

Este flujo garantiza una separación clara entre las capas del sistema.

5.5 Formato de datos (JSON)

La API utiliza JSON como formato estándar para el intercambio de información.

Ejemplo de respuesta:

```
{
 "id": 1,
 "name": "Press banca",
 "muscle": "Pecho"
}
```

Se ha optado por un formato de datos plano, evitando estructuras anidadas complejas. Esta decisión facilita el procesamiento en el frontend y evita problemas de serialización.

5.6 Problemas técnicos y soluciones

Durante el desarrollo de la API se detectaron problemas relacionados con la serialización de entidades con relaciones bidireccionales, lo que provocaba recursividad infinita en las respuestas JSON.

Este problema se solucionó mediante el uso de anotaciones como:

@JsonIgnore

De esta forma, se evita incluir ciertas relaciones en la respuesta, simplificando la estructura del JSON.

6. BASE DE DATOS

6.1 Sistema gestor

Se ha utilizado MySQL como sistema gestor de base de datos relacional debido a su fiabilidad, rendimiento y compatibilidad con herramientas como Spring Boot y Hibernate.

6.2 Diseño de la base de datos

El diseño de la base de datos se basa en un modelo relacional normalizado, donde cada entidad representa una tabla independiente.

Se han utilizado claves primarias para identificar cada registro y claves foráneas para establecer relaciones entre tablas.

6.3 Integridad referencial

La integridad referencial se garantiza mediante el uso de claves foráneas, lo que impide la creación de registros inconsistentes.

Por ejemplo, no es posible crear una rutina asociada a un usuario inexistente.

6.4 Normalización

La base de datos ha sido diseñada siguiendo principios de normalización, lo que permite:

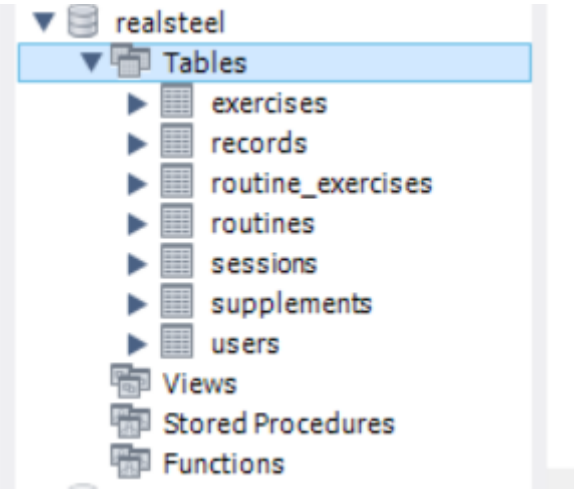
- Reducir la redundancia de datos
- Evitar inconsistencias
- Mejorar la eficiencia de las consultas

6.5 Ejemplo de tablas

Las tablas principales del sistema son:

- users
- routines
- exercises
- records

Cada una de estas tablas está relacionada mediante claves foráneas, lo que permite reconstruir la información completa del sistema.



Esta seria como quedaría las tablas en la base de datos.

7. PRUEBAS

El proceso de pruebas ha sido una parte fundamental en el desarrollo del proyecto RealSteel, ya que ha permitido verificar el correcto funcionamiento del sistema en sus diferentes capas: backend, frontend e integración entre ambos.

Dado que la aplicación sigue una arquitectura cliente-servidor, se han realizado pruebas específicas en cada componente, así como pruebas conjuntas para garantizar la correcta comunicación entre ellos.

7.1 Pruebas de backend

Las pruebas del backend se han centrado en verificar el correcto funcionamiento de la API REST desarrollada con Spring Boot. Para ello, se ha utilizado la herramienta Postman, que permite simular peticiones HTTP y analizar las respuestas del servidor.

A través de estas pruebas se ha podido comprobar que los endpoints definidos responden correctamente a las solicitudes realizadas, tanto en operaciones de consulta (GET) como de creación de datos (POST).

Entre los aspectos verificados destacan:

- La correcta recepción de datos enviados en formato JSON.
- La validación de los datos antes de ser procesados por la aplicación.
- La generación de respuestas adecuadas por parte del servidor.
- El correcto almacenamiento de la información en la base de datos.

Además, estas pruebas han sido clave para detectar errores en la lógica de negocio, como fallos en las relaciones entre entidades o problemas en las consultas realizadas mediante JPA.

El uso de Postman ha permitido iterar rápidamente sobre los endpoints, facilitando la depuración del sistema y asegurando que la API cumple con los requisitos definidos.

7.2 Pruebas de frontend

En el frontend, las pruebas se han centrado en la experiencia de usuario y en la correcta representación de los datos obtenidos desde la API.

Estas pruebas se han realizado directamente sobre la aplicación desarrollada en Flutter, tanto en entorno de desarrollo como en dispositivos reales.

Se han evaluado los siguientes aspectos:

- La correcta visualización de la información en pantalla, asegurando que los datos se muestran de forma clara y ordenada.
- La navegación entre diferentes pantallas de la aplicación, comprobando que el flujo de uso es intuitivo y funcional.
- La interacción del usuario con los distintos elementos de la interfaz, como botones, formularios y listas.
- La gestión de estados, verificando que la aplicación responde adecuadamente a cambios en los datos.

También se han detectado posibles errores relacionados con la carga de datos, como pantallas vacías o problemas al mostrar listas dinámicas, los cuales han sido corregidos durante el proceso de desarrollo.

7.3 Pruebas de integración

Las pruebas de integración han sido especialmente relevantes en este proyecto, ya que permiten validar la comunicación entre el frontend y el backend.

Estas pruebas consisten en comprobar que los datos fluyen correctamente entre ambas capas, desde que el usuario realiza una acción en la aplicación hasta que la información se almacena en la base de datos.

Se han verificado los siguientes aspectos:

- El envío correcto de datos desde la aplicación Flutter hacia la API mediante peticiones HTTP.
- La correcta recepción y procesamiento de dichos datos en el backend.
- La devolución de respuestas en formato JSON coherente con lo esperado por el frontend.

- La correcta inserción y recuperación de datos en la base de datos MySQL.

Durante estas pruebas también se han identificado problemas de compatibilidad entre el formato de datos esperado por Flutter y el generado por la API, lo que ha llevado a ajustar las estructuras JSON para garantizar su correcta interpretación.

Estas pruebas han sido esenciales para asegurar el funcionamiento global del sistema.

8. PROBLEMAS ENCONTRADOS

Durante el desarrollo del proyecto se han identificado diversos problemas técnicos, tanto a nivel de backend como de frontend e integración. La resolución de estos problemas ha permitido mejorar la estabilidad y el funcionamiento general de la aplicación.

8.1 Problemas de conexión entre Flutter y la API

Uno de los principales problemas encontrados ha sido la dificultad para establecer la conexión entre la aplicación Flutter y el backend, especialmente al ejecutar la aplicación en dispositivos móviles reales.

Esto se debe a que, en un entorno local, el uso de direcciones como "localhost" no es válido para dispositivos externos. Como solución, se ha utilizado la dirección IP local del equipo que ejecuta el servidor, permitiendo así la comunicación dentro de la misma red.

8.2 Configuración de direcciones IP

Relacionado con el punto anterior, se han detectado errores derivados de una configuración incorrecta de las direcciones IP.

Para solucionar este problema, se ha identificado la dirección IPv4 del equipo mediante herramientas del sistema (como el comando ipconfig) y se ha configurado correctamente en el código del frontend.

8.3 Problemas de serialización en JSON

Otro problema relevante ha sido la aparición de recursividad infinita en las respuestas JSON generadas por el backend, debido a las relaciones bidireccionales entre entidades en JPA.

Este problema provoca que el servidor intente serializar objetos de forma infinita, generando errores o respuestas inválidas.

La solución adoptada ha sido el uso de anotaciones como @JsonIgnore, que permiten excluir ciertos atributos de la serialización y evitar ciclos en las relaciones.

8.4 Problemas con dependencias en Flutter

Durante el desarrollo del frontend también se han encontrado problemas relacionados con dependencias no instaladas o mal configuradas, especialmente al integrar librerías externas como el paquete HTTP.

Estos problemas se han resuelto mediante la correcta configuración del archivo pubspec.yaml y la ejecución de comandos como "flutter pub get".

8.5 Otros problemas detectados

Además de los anteriores, se han identificado otros inconvenientes menores, como errores en la carga de datos, problemas de navegación entre pantallas o inconsistencias en la visualización de la información.

Todos estos problemas han sido resueltos mediante pruebas iterativas y ajustes en el código.

9. DESPLIEGUE

El despliegue del sistema se ha realizado en un entorno local, lo que permite probar y validar la aplicación durante su desarrollo.

9.1 Ejecución del backend

El backend desarrollado con Spring Boot se ejecuta en un servidor local, accesible mediante la siguiente dirección:

http://localhost:8080

Este entorno permite probar la API de forma rápida y realizar cambios en tiempo real durante el desarrollo.

9.2 Acceso desde dispositivos móviles

Para permitir el acceso desde dispositivos móviles, se utiliza la dirección IP local del equipo que ejecuta el backend:

http://192.168.X.X:8080

De esta forma, el dispositivo móvil puede conectarse al servidor siempre que ambos estén en la misma red.

9.3 Consideraciones de red

Para que el sistema funcione correctamente en este entorno, es necesario cumplir ciertas condiciones:

- El dispositivo móvil y el ordenador deben estar conectados a la misma red Wi-Fi.
- El puerto utilizado (8080) debe estar accesible.
- El firewall del sistema no debe bloquear la conexión.

9.4 Posibles mejoras en despliegue

En futuras versiones, el sistema podría desplegarse en un servidor remoto o en la nube, permitiendo el acceso desde cualquier ubicación sin necesidad de configuración de red local.

10. LIMITACIONES ACTUALES

El proyecto RealSteel se encuentra en una fase de producto mínimo viable, lo que implica la existencia de ciertas limitaciones que deberán ser abordadas en futuras versiones.

10.1 Falta de autenticación segura

Actualmente, el sistema no implementa un mecanismo de autenticación basado en tokens (como JWT), lo que limita la seguridad de la aplicación.

10.2 Funcionalidades limitadas

El sistema carece de ciertas funcionalidades avanzadas, como estadísticas detalladas, gráficos de progreso o recomendaciones personalizadas.

10.3 Dependencia de conexión

La aplicación requiere conexión a internet para funcionar, ya que no dispone de un modo offline.

10.4 Gestión básica de errores

El manejo de errores es limitado y no proporciona mensajes detallados al usuario, lo que puede afectar a la experiencia de uso.

11. PRÓXIMOS PASOS

El desarrollo futuro de la aplicación contempla diversas mejoras orientadas a ampliar sus funcionalidades y mejorar la experiencia de usuario.

11.1 Implementación de autenticación con JWT

Se plantea incorporar un sistema de autenticación basado en tokens, que permita mejorar la seguridad del sistema y proteger los datos del usuario.

11.2 Desarrollo de estadísticas

Se prevé implementar un sistema de análisis de datos que permita al usuario visualizar su progreso mediante gráficos y métricas.

11.3 Notificaciones

La incorporación de notificaciones permitirá recordar al usuario sus entrenamientos y mejorar la constancia.

11.4 Mejora de la interfaz

Se trabajará en la mejora de la interfaz de usuario, optimizando la navegación y el diseño visual.

11.5 Escalabilidad del sistema

El sistema está diseñado para permitir futuras ampliaciones, como la integración con dispositivos externos o la incorporación de nuevas funcionalidades.